

Funções

Programação Imperativa

Prof. Pedro Rocha Horchulhack

Contato: p.horchulhack@pucpr.br

Agenda

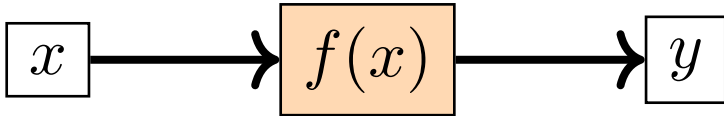
1. O que são funções?
2. Passagem de parâmetros

Agenda

1. O que são funções?
2. Passagem de parâmetros

O que são funções?

- Além dos módulos em Rust, podemos dividir nosso código em pequenas sequências de código



O que são funções?

Assinatura de uma função

- Conhecemos a função `main()`;
- Nela não vimos nenhum **parâmetro** nem **tipo de retorno**;
- A assinatura da função informa ao compilador:
 - Nome;
 - Conjunto de parâmetros (se houver);
 - Tipo de retorno (se houver).

```
fn NOME_DA_FUNCAO(parametro1: TIPO, parametro2: TIPO2) -> TIPO DO RETORNO {  
    // Código  
}
```

O que são funções?

Assinatura de uma função

Discuta com os colegas

Qual a assinatura das funções abaixo?

```
fn funcao1() { }  
  
fn funcao2(parametro1: i32, parametro2: i32) { }  
  
fn funcao3(x: String, y: String) -> String { }  
  
fn funcao4(x: String, y: String) -> bool { }
```

O que são funções?

Retornos

- O retorno de uma função é o resultado que ela produz e que **pode ser armazenado**;
- As funções podem ter tipos de retorno ou a ausência deles;

```
fn soma_inteiros(x: i32, y: i32) -> i32 {  
    // Implementar lógica  
}
```

O que são funções?

Retornos

- Indicamos o **valor** que queremos retornar de duas maneiras:
 - Diretiva `return`;
 - Última linha da função **SEM PONTO E VÍRGULA**.

```
fn soma_inteiros(x: i32, y: i32) -> i32 {  
    return x + y;  
}
```

```
fn soma_inteiros(x: i32, y: i32) -> i32 {  
    x + y  
}
```

O que são funções?

Retornos

```
fn soma_inteiros(x: i32, y: i32) -> i32 {  
    let soma: i32 = x + y;  
  
    if soma > 10 {  
        return 1;  
    } else {  
        return 100;  
    }  
}
```

```
fn soma_inteiros(x: i32, y: i32) -> i32 {  
    let soma: i32 = x + y;  
  
    if soma > 10 {  
        return 1;  
    }  
    100  
}
```

O que são funções?

Atividade

1. Crie uma função que receba dois parâmetros do tipo inteiro não-assinado de 8 bits e retorne se são iguais;
2. Crie uma função que receba um número e retorne o seu quadrado;
3. Crie uma função que receba um inteiro assinado de 32 bits e calcule todos os números pares, múltiplos de 3, até o número fornecido como parâmetro.
4. Crie uma função que retorne o *input* do usuário, convertendo-o para o tipo i64.

Agenda

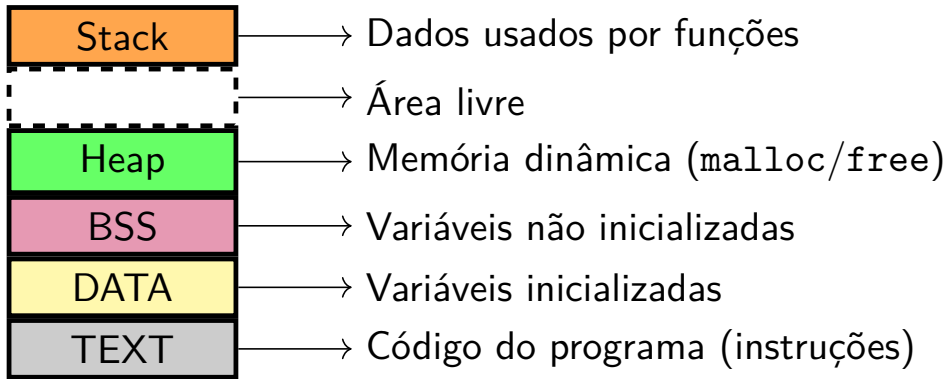
1. O que são funções?
- 2. Passagem de parâmetros**

Passagem de parâmetros

- Funções possuem um escopo, conjunto de parâmetros e um retorno;
- Principais pontos:
 - Como uma função recebe os parâmetros?
 - Uma função ocupa espaço na memória?
 - Como o código sabe para onde retornar após a chamada de uma função?
- Como discutido anteriormente, todos os códigos possuem **regiões de memória** para propósitos específicos.

Passagem de parâmetros

Regiões de Memória



Passagem de parâmetros

Armazenamento em funções – sem parâmetros

```
fn pilha() {  
    let b = 30;  
    let c = 40;  
}  
  
fn main() {  
    let a = 20;  
    pilha();  
}
```

Tabela: Stack (pilha)

Endereço	Nome	Valor

Passagem de parâmetros

Armazenamento em funções – com parâmetros

```
fn pilha(d: i32) {  
    let b = 30;  
    let c = 40;  
}  
  
fn main() {  
    let a = 20;  
    pilha(50);  
}
```

Tabela: Stack (pilha)

Endereço	Nome	Valor

Passagem de parâmetros

Passagem por cópia

- Quando especificamos os parâmetros de uma função, existem dois mecanismos que podemos aplicar
 - Cópia de valores
 - Referência à valores
- A maioria das linguagens de programação permitem a passagem de parâmetros copiando valores

Passagem de parâmetros

Passagem por cópia

- Nesse mecanismo os parâmetros são alocados como variáveis locais dentro do escopo da função, de modo que os valores passados como parâmetros sejam copiados às variáveis de parâmetro

```
fn dobro(i: i32) -> i32 {  
    i * 2  
}
```

Passagem de parâmetros

Passagem por cópia

```
fn dobro(i: i32) -> i32 {  
    i * 2  
}  
  
fn main() {  
    let x = 3;  
    let y = dobro(x);  
    println!("Dobro de {} é {}", x, y);  
    // Dobro de 3 é 6  
}
```

Passagem de parâmetros

Passagem por referência

- Aqui passamos uma **referência**, ou seja, a localização na memória daquela variável em específico. Acessamos **diretamente** a variável ao invés de copiarmos o valor para uma nova variável.
- Uma referência à variável é análogo à um endereço na memória, ou seja, a sua exata localização.
- No caso anterior nós copiávamos valores, implicando em maior consumo de memória. Referência à valores pode nos ajudar a “economizar” recursos e a reutilizar variáveis por maiores períodos.
- Associado à isso, teremos dois operadores:
 - **&**: Indica o endereço da memória (referência) à uma variável
 - ***** : Além de ser um operador aritmético (multiplicação), podemos utilizá-lo para acessarmos o valor do endereço da memória (referência)

Passagem de parâmetros

Passagem por referência

```
fn dobro(i: i32) -> i32 {  
    i * 2  
}  
  
fn main() {  
    let x = 3;  
    let y = dobro(x);  
    println!("Dobro de {} é {}", x, y);  
    // Dobro de 3 é 6  
}
```

Passagem de parâmetros

Passagem por referência

```
fn dobro(i: &mut i32) -> i32 {  
    *i = *i * 2  
}
```

```
fn main() {  
    let mut x = 3;  
    print!("Dobro de {} ", x);  
    dobro(&mut x);  
    println!("é {}", x);  
}
```

Passagem de parâmetros

Verificando o endereço de uma variável

```
fn main() {  
    let x: u32 = 10;  
    let y: String = String::from("Olá");  
  
    println!("Endereço de x={:p}", &x);  
    println!("Endereço de y={:p}", &y);  
}
```

Passagem de parâmetros

Atividade

1. Escreva uma função que receba dois valores, que sejam mutáveis e troque seus valores. Utilize referências para realizar a troca.
2. Escreva uma função que calcule a média de um vetor de números inteiros. A função deve receber uma referência ao vetor como parâmetro e retornar a média.
3. Crie uma função que verifique se uma palavra é um palíndromo, ou seja, ela é a mesma quando é lida de trás para frente. A função deve receber uma referência à uma string como parâmetro e retornar um booleano indicando se é um palíndromo ou não.